

CMSC6950 Homework #2

Isaac Adoboe - ieadoboe@mun.ca

Question 1

Numpy

[Numpy GitHub link](#)

Numpy is a popular and fundamental package used for mathematics, statistics and scientific computing in Python. It supports multi-dimensional arrays and high-performance mathematical operations. Numpy's code quality and standards affect lots of other packages that adopt it. It becomes extremely important that the write good code for efficiency and usability.

Numpy exemplifies good code practices. The codebase is open and well-documented with seperate website with examples and guides. First, most of the functions have meaningful names (mostly relatable from mathematics), e.g. `tensor_dot` which computes the tensor dot product. In `numpy.zeros_like` in `numpy/core/numeric.py` the had over 60 lines of doctring explaining parameters and outputs returned with usage examples. All functions follow a consistent coding style across the whole codebase.

The library is demonstrably runnable and correct featuring thousands unit tests as seen in the `numpy/core/tests/` directory with over 60 seperate test files. Each test has a meaningful name describing exactly what it verifies. Functions are appropriately modular—for instance, `numpy.mean()` delegates to smaller helper functions for axis validation and data type handling rather than doing everything in one massive function.

Question 2

```
In [1]: import math as m

def cot(x):
    if x == 0:
        print("Warning: cot(x) not defined for x=0")
    if abs(x) >= m.pi:
        print("Warning: Input might be out of range")
```

```

assert -m.pi < x < m.pi, "x must be in the range (-pi, pi)."
assert x != 0, "cot(x) is not defined for x=0."

return m.cos(x)/m.sin(x)

```

```

In [4]: # Test 1
        cot(2*m.pi)

```

Warning: Input might be out of range

```

-----
AssertionError                                Traceback (most recent call l
ast)
Cell In[4], line 2
      1 # Test 1
----> 2 cot(2*m.pi)

Cell In[1], line 9, in cot(x)
      6 if abs(x) >= m.pi:
      7     print("Warning: Input might be out of range")
----> 9 assert -m.pi < x < m.pi, "x must be in the range (-pi, pi)."
     10 assert x != 0, "cot(x) is not defined for x=0."
     12 return m.cos(x)/m.sin(x)

AssertionError: x must be in the range (-pi, pi).

```

```

In [74]: # Test 2
         cot(0)

```

Warning: cot(x) not defined for x=0

```

-----
AssertionError                                Traceback (most recent call l
ast)
Cell In[74], line 2
      1 # Test 2
----> 2 cot(0)

Cell In[70], line 10, in cot(x)
      7     print("Warning: Input might be out of range")
      9 assert -m.pi < x < m.pi, "x must be in the range (-pi, pi)."
----> 10 assert x != 0, "cot(x) is not defined for x=0."
     12 return m.cos(x)/m.sin(x)

AssertionError: cot(x) is not defined for x=0.

```

```

In [72]: # Test 3
         cot(m.pi/4)

```

Out[72]: 1.0000000000000002

```

In [73]: # Test 4

```

```
cot(-m.pi/3)
```

```
Out[73]: -0.577350269189626
```

Question 3

```
In [ ]: def csc(x):
        """
        Calculate csc(x) = 1/sin(x) and returns the value when well-define
        Prints error message when sin(x) = 0 or x is not a real number.
        """
        try:
            result = 1 / m.sin(x)
            return result
        except ZeroDivisionError:
            print("Error: csc(x) is undefined when sin(x) = 0")
            return None
        except TypeError:
            print("Error: Input must be a real number")
            return None
```

```
In [12]: # Test
         csc(m.pi)
```

```
Out[12]: 8165619676597685.0
```

```
In [ ]: # Test 2
         csc(0)
```

```
Error: csc(x) is undefined when sin(x) = 0
```

```
In [11]: # Test 3
         csc("1j")
```

```
Error: Input must be a real number
```

Question 4

a. Explain the difference between the two functions

```
@x_times_f
@add_constant(4)
def f1(x):
    return x
```

```
@add_constant(4)
@x_times_f
def f2(x):
    return x
```

What mathematical functions are being computed in the two cases?

Answer

When using decorators, the functions wrap bottom to top. What do I mean by this?

```
final_function = decorator2(decorator1(func))
```

For the first function,

```
`x_times_f(add_constant(f1(x)))`  
`x_times_f(add_constant(x))`  
`x_times_f(x+4)`  
`def result(x): return x * (x+4)`  
`x**2 + 4x`
```

$$x^2 + 4x$$

For the second function, since the arrangement changed the way the functions are applied will change as well and *may* result in a different solution:

```
`add_constant(x_times_f(f2(x)))`  
`add_constant(x_times_f(x))`  
`add_constant(x * x)`  
`x**2 + 4`
```

$$x^2 + 4$$

b. Consider the code

```
@add_constant(1)  
@x_times_f  
@add_constant(3)  
def q(x):  
    return x
```

What mathematical function, $q(x)$, does this compute?

```
`add_constant(x_times_f(add_constant(q(x))))`  
`add_constant(x_times_f(add_constant(x)))`  
`add_constant(x_times_f(x+3))`  
`add_constant(x*(x+3))`  
`x*(x+3)+1`
```

$$q(x) = x(x + 3) + 1 = x^2 + 3x + 1$$

c. Compose the two decorators to write a function that evaluates the polynomial $p(x) = x^3 + 5x^2 - 2x + 3$. Hand in a transcript of your Jupyter notebook, showing your function definition and some tests that it produces the correct values. No points will be given for functions that evaluate $p(x)$ directly, only for functions that use the decorators.

```
In [20]: class add_constant(object):
    def __init__(self, constant):
        self.constant = constant
    def __call__(self, f):
        def result(x):
            return f(x) + self.constant
        return result

    def x_times_f(f):
        def result(x):
            return x * f(x)
        return result

    @add_constant(3)
    @x_times_f
    @add_constant(-2)
    @x_times_f
    @add_constant(5)
    def p(x):
        return x
```

```
In [21]: # Test 1, we know if x = 0, p(0) = 3
p(0)
```

```
Out[21]: 3
```

```
In [22]: # Test 2, if x = 1, we expect p(1) = 7
p(1)
```

```
Out[22]: 7
```

```
In [23]: # Test 3, if x = -2, we expect p(-2) = 19
p(-2)
```

```
Out[23]: 19
```

```
In [ ]:
```